Neural Network for Recruitment Outcome Prediction

Prepared By: Nimani Thathsarani

Table of Contents

1. Objective 3
2. Dataset 3
3. Architecture Experiments 4
4. Preventing Overfitting 5
5. Final Model Evaluation 6
6. Conclusion 7

1. Objective

The main objective of this individual task is to develop a neural network for classifying job applicants into three possible recruitment outcomes: Hired, Pending, or Rejected**.** The model uses candidate information such as position applied for, years of experience, technical assessment score, and interview score to predict the recruitment outcome.

The task also aims to investigate how different neural network architectures affect classification performance. Multiple architectures with different numbers of hidden layers and neurons are trained and compared based on their training and validation performance.

A further objective is to identify and reduce overfitting**,** which can occur when a neural network performs well on training data but poorly on unseen data. Techniques including Dropout, L2 regularization, and Early Stopping are applied to improve the model's ability to generalize. Finally, the selected model is evaluated on an unseen test dataset to determine its overall classification performance.

2. Dataset

The Recruitment dataset (200 candidates) was used. It was the cleanest and most balanced dataset available for a classification task: three roughly even classes and features that show a clear relationship with the outcome (hired candidates have noticeably higher average tech and interview scores than pending or rejected ones).

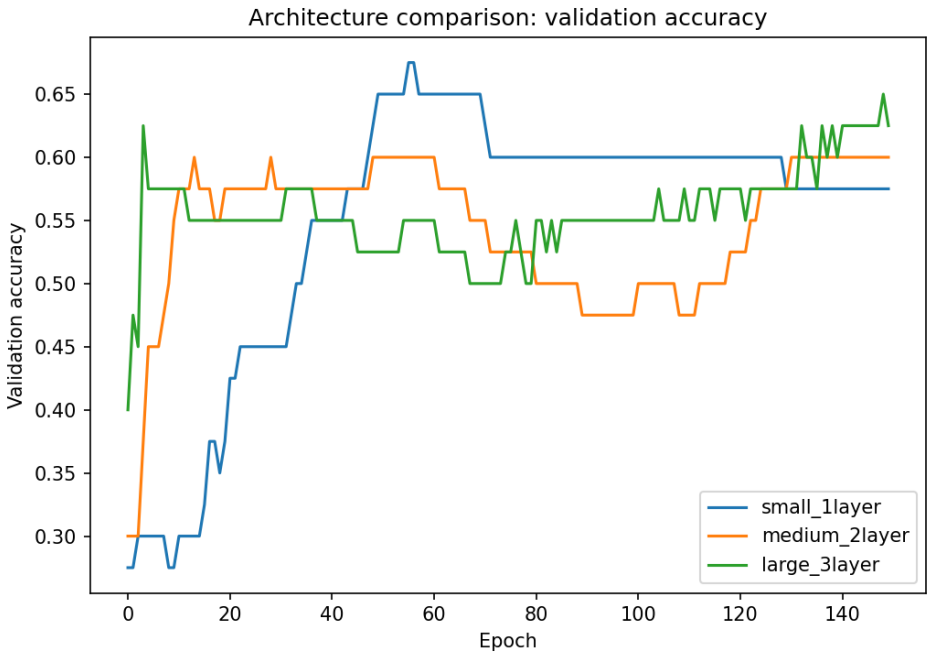
Feature	Type	Description
Position	Categorical	DS / SE / QA — role applied for
Experience	Numeric	Years of relevant experience (0–10)
Tech_Score	Numeric	Technical assessment score (40–100)
Interview	Numeric	Interview score (41–100)
Status (target)	Categorical	Hired / Pending / Rejected

Position was one-hot encoded, and the three numeric features were standardized (zero mean, unit variance), giving an input vector of 6 values per candidate. The data was split into training (120), validation (40), and test (40) sets using a stratified 60/20/20 split, so all three classes are represented proportionally in every split.

3. Architecture Experiments

Three architectures of increasing size were trained under identical conditions (Adam optimizer, categorical cross-entropy loss, 150 epochs, batch size 16) to see how capacity affects the fit:

Architecture	Hidden Layers	Best val. Acc.	Train Acc.	Val. Acc.	Gap
small_1layer	[8]	0.675	0.733	0.575	0.158
medium_2layer	[32, 16]	0.600	0.900	0.600	0.300
large_3layer	[64, 32, 16]	0.650	1.000	0.625	0.375

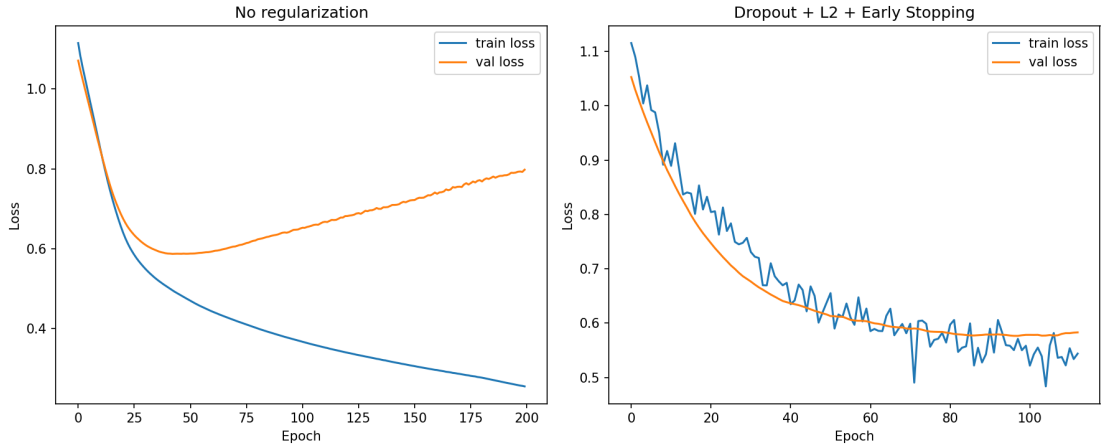


As the network gets larger, training accuracy climbs (the 3-layer model reaches 100% on the training set) but validation accuracy does not improve at the same rate the gap between train and validation accuracy grows with model size. This is the classic overfitting pattern: a larger network has enough capacity to memorize the 120 training examples rather than learn a generalizable pattern. The medium (32, 16) architecture was carried forward as the base for the next stage, since it balances capacity and stability better than the largest model.

4. Preventing Overfitting

The medium architecture was trained twice: once with no regularization, and once with three overfitting-prevention techniques combined:

- Dropout (rate 0.3) after each hidden layer, to prevent co-adaptation of neurons
- L2 weight regularization (λ = 1e-3), to keep weights small
- Early stopping on validation loss (patience = 15 epochs), restoring the best-performing weights



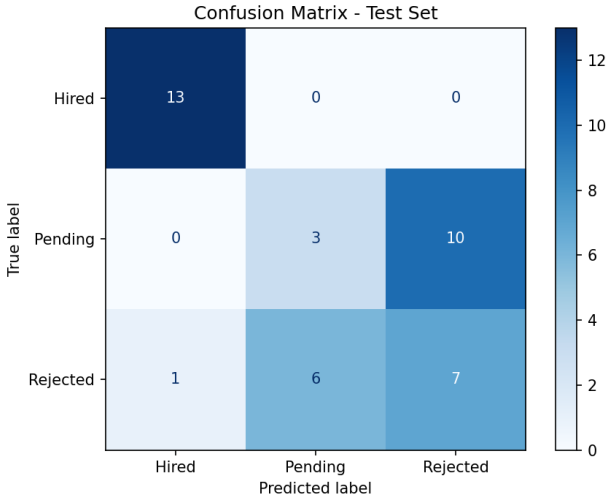
Model	Train acc.	Val. Acc.	Gap
No regularization	0.892	0.600	0.292
Dropout + L2 + Early Stopping	0.667	0.575	0.092

The unregularized model's training loss keeps falling while validation loss flattens and then rises after roughly 30–40 epochs a textbook overfitting curve. With dropout, L2 regularization, and early stopping, the train–val gap shrinks from about 0.29 to about 0.09, and training stopped automatically after 113 epochs once validation loss stopped improving, instead of continuing to overfit for the full 200 epochs. The regularized model was kept as the final model it generalizes more honestly, even though its raw training accuracy is lower.

5. Final Model Evaluation

The regularized model was evaluated once on the held-out test set (40 candidates, never seen during training or model selection)

Metric	Value
Test accuracy	57.5%
Test loss	0.570



The model identifies Hired candidates very reliably (high precision and recall), which makes sense given the clear score separation seen in the data. Pending and Rejected are harder to tell apart this matches the raw data, where their average Tech_Score and Interview values are close to each other. With only 200 total candidates, this is a reasonable result; more data, especially for the Pending/Rejected boundary, would likely improve this further.

6. Conclusion

In this individual task, a neural network was successfully developed to classify job candidates into Hired, Pending, and Rejected categories. The recruitment dataset was preprocessed using one-hot encoding for the categorical position feature and standardization for the numerical features before being divided into training, validation, and test sets.

Three different neural network architectures were experimented with to examine the effect of model complexity. The results showed that increasing the size of the network improved training accuracy but also increased the difference between training and validation performance. In particular, the largest architecture achieved 100% training accuracy while achieving only 62.5% validation accuracy**,** demonstrating a clear case of overfitting.

To address this problem, the medium architecture was further trained using Dropout, L2 weight regularization, and Early Stopping. These techniques reduced the training-validation accuracy gap from approximately 29.2% to 9.2%**,** showing that the regularized model was less prone to overfitting and had better generalization behavior.

The final model achieved 57.5% accuracy on the unseen test dataset. Although the overall accuracy leaves room for improvement, the experiment successfully demonstrated the complete neural network development process, including preprocessing, architecture comparison, overfitting detection, regularization, and final model evaluation. With a larger dataset and additional features, the model could potentially achieve better performance, particularly in distinguishing between the Pending and Rejected outcomes.

Overall, this task demonstrated that selecting an appropriate neural network architecture and applying suitable regularization techniques are important for developing a model that generalizes effectively to unseen recruitment data.